САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Фёдоров Иван

Группа К3340

Проверил:
Добряков Д. И.

Санкт-Петербург

1 Задача работы

Разделить монолитное REST API RentEase, разработанное в Лабораторной работе 1, на независимые микросервисы согласно архитектуре из ДЗ4. Для каждого сервиса необходимо выделить собственную базу данных, сохранить пользовательский сценарий аренды и обеспечить межсервисное взаимодействие через внутренние API.

2 Ход работы

1. Проанализирована предметная область монолита: пользователи и роли, объявления, избранное, бронирования, договоры, платежи, сообщения и отзывы.
2. Определены границы сервисов и правило database-per-service. Между сервисами не используются внешние SQL-связи и внешние ключи.
3. Создан родительский Maven-проект с общим модулем common и отдельными модулями микросервисов.
4. Вынесены Identity, Property, Booking, Contract, Communication и Review Service, а также API Gateway.
5. Для каждого сервиса добавлены JPA-сущности, Spring Data репозитории и независимая миграция Flyway V1.
6. Реализованы JWT для пользователей и короткоживущие сервисные JWT для внутренних вызовов.
7. Добавлена сквозная проверка регистрации, входа, объявления, сообщения, бронирования, подтверждения и автоматического создания договора.

3 Архитектура решения

Клиент обращается к единой точке входа API Gateway на порту 8080. Gateway маршрутизирует публичные URL к нужному сервису и не публикует внутренние endpoint'ы. Сервисы запускаются независимо и имеют собственные порты и базы.

Компонент	Порт	Ответственность	База
API Gateway	8080	Маршрутизация публичного API	Нет
Identity Service	8081	Регистрация, вход, роли, JWT, refresh	identity

Property Service	8082	Объявления и избранное	property
Booking Service	8083	Бронирования и статусы	booking
Contract Service	8084	Договоры и платежи	contract
Communication Service	8085	Сообщения и диалоги	communication
Review Service	8086	Отзывы	review

4 Разделение базы данных

Каждый сервис изменяет только собственную базу через свои репозитории. UUID-ссылки на сущности другого сервиса хранятся без внешнего ключа и проверяются внутренним REST-вызовом с сервисным bearer-токеном.

База	Основные таблицы	Внешние UUID-ссылки
identity	users, user_roles, refresh_tokens	Нет
property	properties, rental_conditions, property_photos, favorites	landlord_id, user_id
booking	bookings, booking_status_history	tenant_id, property_id
contract	rental_contracts, payments	booking_id, property_id, tenant_id, landlord_id
communication	messages	sender_id, receiver_id, property_id
review	reviews	contract_id, author_id, property_id

Миграции находятся в src/main/resources/db/migration каждого сервиса. В локальной конфигурации используется H2, а параметр DB_URL позволяет подключить отдельную PostgreSQL-базу для каждого сервиса.

5 Межсервисное взаимодействие

Инициатор	Получатель	Назначение
Booking Service	Property Service	Проверить активность объекта, владельца и цену
Booking Service	Contract Service	Создать договор после подтверждения бронирования
Communication Service	Identity Service	Проверить получателя сообщения
Communication Service	Property Service	Проверить объект сообщения
Review Service	Contract Service	Проверить право оставить отзыв

Основной поток бронирования: tenant регистрируется и входит через Identity Service, создает бронирование через Gateway, Booking Service получает снимок объекта из Property Service, landlord подтверждает заявку, после чего Booking Service вызывает внутренний POST /internal/contracts и Contract Service идемпотентно создает договор.

Публичный маршрут диалога /api/v1/messages/dialogs/{user_id}/{property_id} сохранен для совместимости с API Лабораторной работы 1 и коллекцией Postman из ДЗ3.

6 Реализованные API

Сервис	Основные публичные endpoint'ы	Внутренние endpoint'ы
Identity	POST /auth/register, POST /auth/login, POST /auth/refresh, GET /users/me	GET /internal/users/{id}/status
Property	GET/POST /properties, GET/POST /favorites	GET /internal/properties/{id}/summary, ownership

Booking	GET/POST /bookings, PATCH /bookings/{id}/status	Вызовы Property и Contract
Contract	GET /contracts	POST /internal/contracts, GET review-eligibility
Communication	GET/POST /messages, GET /messages/dialogs/...	Вызовы Identity и Property
Review	GET/POST /reviews	Вызов Contract review-eligibility

Спецификация внутренних контрактов в формате OpenAPI сохранена в hw4/internal-openapi.yaml. Для каждого сервиса доступна Swagger UI по адресу http://localhost:808X/docs.

7 Проверка работы

Сборка выполняется из каталога БР1.1/Lab2 командой:

```
mvn clean verify
```

Итоговая чистая сборка текущей версии завершилась BUILD SUCCESS для всех девяти модулей: common, identity-service, property-service, booking-service, contract-service, communication-service, review-service и api-gateway.

Для интеграционной проверки после запуска сервисов используется воспроизводимый сценарий:

```
.\smoke-test.ps1
```

Smoke-тест через Gateway проверяет регистрацию landlord и tenant, получение JWT, создание объявления, сообщение и диалог, создание и подтверждение бронирования, автоматическое создание договора, а также ответы 401 без токена, 403 при нарушении роли, 409 при пересечении дат и 400 при неверном диапазоне дат.

Проверка	Ожидаемый результат	Фактический результат
mvn clean verify	BUILD SUCCESS	Пройдено
/api/v1/health на 8080-8086	healthy	Пройдено
Основной пользовательский сценарий	Создано бронирование и договор	Пройдено
Отсутствует access token	401	Пройдено

Неверная роль	403	Пройдено
Пересечение дат	409	Пройдено
Неверные даты	400	Пройдено
Swagger /docs на 8080-8086	HTTP 200	Пройдено

В Maven-проекте пока нет автоматических JUnit-тестов: в выводе присутствует No tests to run. Поэтому бизнес-сценарий проверяется интеграционным smoke-тестом; добавление unit- и contract-тестов является следующим улучшением качества.

8 Рекомендуемые скриншоты для защиты

- Вывод Maven с BUILD SUCCESS и сводкой девяти модулей.
- Ответы GET /api/v1/health на портах 8080-8086.
- Консоль с результатом smoke-test.ps1 и PASS для кодов 401, 403, 409 и 400.
- Swagger UI одного из сервисов, например http://localhost:8080/docs, с раскрытым endpoint'ом.
- Лог Booking Service с вызовами Property Service и Contract Service.

9 Вывод

В ходе работы монолитный RentEase разделен на шесть предметных микросервисов и API Gateway. Каждый сервис имеет собственную модель данных, репозитории и Flyway-миграцию. Внутренние зависимости реализованы через REST и сервисные JWT, а внешний пользовательский сценарий сохранен через единый Gateway. Сборка, health-проверки, Swagger и сквозной smoke-тест подтверждают работоспособность реализованной архитектуры. Для дальнейшего повышения качества требуется добавить автоматические unit- и contract-тесты, а для следующих работ курса можно подключить брокер сообщений и контейнеризацию.